Online House Rental Management System (OHRMS)

Software Requirements Specification (SRS)

Version: 1.0.0
Date: 17-02-2026
Prepared For: CodeCrafters Team
Prepared By: Nusrat Jahan, Maimuna Nowshen Mahima, Maksuda Akter Mohona, Rukaiya Tamanna

---

Table of Contents

Table of Contents i
1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Project Scope 1
1.4 References 1
2. Overall Description 2
2.1 Product Perspective 2
2.2 User Classes and Characteristics 2
2.3 Operating Environment 2
2.4 Design and Implementation Constraints 2
2.5 Assumptions and Dependencies 3
3. System Features 3
3.1 User Authentication and Profile Management 3
3.1.1 Description 3
Secure registration and login system. 3
3.1.2 Stimulus/Response Sequences 3
3.1.3 Functional Requirements 3
3.2 Property Listing Management 3
3.2.1 Description 3
Hosts manage rental properties. 3
3.2.2 Stimulus/Response Sequences 3
3.2.3 Functional Requirements 3
3.3 Advance Search and Discovery (The Guest Engine) 4
3.3.1 Description 4
A high-performance search system allowing guests to find homes. 4
3.3.2 Stimulus/Response Sequences 4
3.3.3 Functional Requirements 4
3.4 Booking and Financial Logic (The Business Core) 4
3.4.1 Description 4
Handles the transaction and ensures the system remains profitable. 4
3.4.2 Stimulus/Response Sequences 4
3.4.3 Functional Requirements 4

3.5 Reviews and Ratings 5
3.5.1 Description 5
Handles guests reviews and appear on every house. 5
3.5.2 Stimulus/Response Sequences 5
3.5.3 Functional Requirements 5
3.6 Admin Management 5
3.6.1 Description 5
Handles all users across the system. 5
3.6.2 Stimulus/Response Sequences 5
3.6.3 Functional Requirements 5
4. System Architecture 5
4.1 System Diagram 5
4.2 Map 6
5. Data Requirements 7
5.1 Logical Data Model 7
5.1.1 Main Entities 7
5.1.2 Relationships 7
5.2 Data Dictionary 7
5.2.1 User Table 7
5.2.2 Listings Table 7
5.3 Database Schema 8
5.4 Reports 8
5.5 Data Acquisition, Integrity, Retention, and Disposal 8
6. External Interface Requirements 9
6.1 User Interfaces 9
6.1.1 Admin Dashboard UI 9
6.2 Software Interfaces 9
6.3 Hardware Interfaces 10
6.4 Hardware Interfaces 10
6.5 Communications Interfaces 10
7. Quality Attributes 10
7.1 Usability 10
7.2 Performance 10
7.3 Security 10
7.4 Reliability 10
7.5 Scalability 10
8. Internationalization and Localization Requirements 11
9. Other Requirements 11
10. Future Improvements 11
10.1 Mobile Application 11
10.2 Integrated Online Payment Gateway 11
10.3 Real-Time Messaging System 11
Appendix A: Glossary 11
Appendix B: Analysis Models 12

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements of the **Online House Rental Management System (OHRMS)**.

The system connects property owners (**Hosts**) with renters/travelers (**Guests**) through a web-based platform. It defines system behavior, business logic, database structure, and operational constraints.

This document is intended for:

- Developers (MERN/PERN stack)
- Database designers
- Testers
- Project evaluators

1.2 Document Conventions

- FR-x → Functional Requirement
- NFR-x → Non-Functional Requirement
- Bold text → Important system components or database tables
- All monetary values are assumed in local currency unless specified

1.3 Project Scope

The system is a simplified house rental system. It allows house owners to list properties, set prices, and manage bookings. Guests can search for properties, view details, and book stays. The system handles the "service fee" business model where the platform takes a percentage of the transaction, ensuring profit for both the host and the platform provider.

- Hosts to list rental properties
- Guests to search and book properties
- Admin to monitor system activities
- Platform to earn commission via service fee model

1.4 References

- IEEE Std 830-1998 – Software Requirements Specification
- MERN Stack Documentation
- MySQL 8.0 Documentation
- BCrypt Password Security Documentation

2. Overall Description

2.1 Product Perspective

This is a standalone web application. The frontend is built with **React**, providing a responsive UI. The backend uses **Node.js** with Express to handle API requests, and **SQL (MySQL via XAMPP)** serves as the relational database.

The system follows RESTful API architecture.

2.2 User Classes and Characteristics

2.2.1 Guest

- Registers/login
- Searches properties
- Makes bookings
- Leaves reviews
- Moderate technical knowledge

2.2.2 Host

- Creates property listings
- Sets price & availability
- Manages bookings
- Views earnings dashboard

2.2.3 Admin

- Manages users
- Approves/rejects listings
- Views revenue analytics
- High-level system access

2.3 Operating Environment

- Client: Chrome, Firefox, Safari
- Server: Node.js Runtime
- Database: MySQL 8.0
- OS: Windows/Linux/Mac supported
- Internet required

2.4 Design and Implementation Constraints

- Must use MySQL database
- Password must be hashed using BCrypt
- REST API architecture required
- SQL injection prevention mandatory

- Responsive UI (mobile & desktop)

2.5 Assumptions and Dependencies

- Users have internet connection
- Payment gateway APIs available
- Email service for notifications available
- Database server running properly

3. System Features

3.1 User Authentication and Profile Management

3.1.1 Description

Secure registration and login system.

3.1.2 Stimulus/Response Sequences

- **Action:** User signs up with email and password.
- **Response:** System validates data, hashes the password, and creates a record in the **Users** table.

3.1.3 Functional Requirements

- **FR-1:** Users can register as Guest or Host.
- **FR-2:** System shall hash passwords using BCrypt.
- **FR-3:** Users shall login using email and password.
- **FR-4:** Users can edit profile details.
- **FR-5:** Admin can deactivate user accounts.

3.2 Property Listing Management

3.2.1 Description

Hosts manage rental properties.

3.2.2 Stimulus/Response Sequences

- **Action:** User signs up with email and password.
- **Response:** System validates data, hashes the password, and creates a record in the **Users** table.

3.2.3 Functional Requirements

- **FR-6**: Host can create property listing.
- **FR-7**: Host can upload multiple images (stored as URLs).
- **FR-8**: Host can set weekday & weekend prices.

- **FR-9**: Host can update availability calendar.
- **FR-10**: Host can deactivate listing without deleting it.

3.3 Advance Search and Discovery (The Guest Engine)

3.3.1 Description

A high-performance search system allowing guests to find homes.

3.3.2 Stimulus/Response Sequences

- **Action:** Guest applies a "Price" and "Location" filter.
- **Response:** Node.js executes a SELECT query with WHERE clauses and BETWEEN operators.

3.3.3 Functional Requirements

- **FR-11:** Search by city or zip code.
- **FR-12:** Filter by price range (BETWEEN operator).
- **FR-13:** Filter by amenities (Wi-Fi, AC, Kitchen).
- **FR-14:** Filter by Super host status.
- **FR-15:** System shall calculate total stay cost in real-time.

3.4 Booking and Financial Logic (The Business Core)

3.4.1 Description

Handles the transaction and ensures the system remains profitable.

3.4.2 Stimulus/Response Sequences

- **Action:** Guest applies a "Price" and "Location" filter.
- **Response:** Node.js executes a SELECT query with WHERE clauses and BETWEEN operators.

3.4.3 Functional Requirements

- **FR3.1:** Search by City/Zip code
- **FR3.2:** Filter by "Superhost" status or specific amenities (WiFi, Kitchen, AC).
- **FR-16:** Guest can select check-in and check-out date.
- **FR-17:** System shall check availability before booking.
- **FR-18:** System calculates:
    - Total nights
    - Base price
    - Service fee
- Final payable amount
- **FR-19:** System shall store booking record.
- **FR-20:** Platform shall deduct commission automatically.

- **FR-21:** Host dashboard shows earnings report.
- **FR-22:** Real-time calculation of total stay cost based on selected date ranges.

3.5 Reviews and Ratings

3.5.1 Description

Handles guests reviews and appear on every house.

3.5.2 Stimulus/Response Sequences

- **Action:** Guest gives a review between 1 to 5 stars with comments.
- **Response:** Will appear in the house review section.

3.5.3 Functional Requirements

- **FR-23:** Guest can leave rating after stay completion.
- **FR-24:** Rating must be between 1–5.
- **FR-25:** Average rating auto-calculated.

3.6 Admin Management

3.6.1 Description

Handles all users across the system.

3.6.2 Stimulus/Response Sequences

- **Action:** Admin can active/inactive any user and can restrict any house from the admin application.
- **Response:** Will be applicable to affected guest and houses.

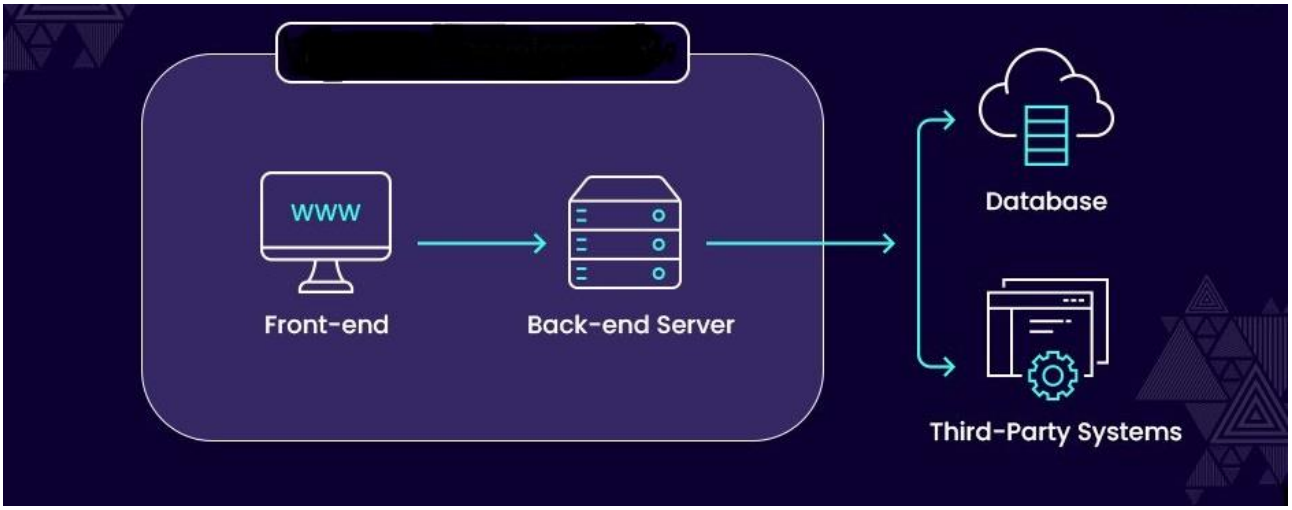
3.6.3 Functional Requirements

- **FR-25:** Admin can view all users.
- **FR-26:** Admin can approve/reject listings.
- **FR-27:** Admin can view total platform revenue.
- **FR-28:** Admin can suspend fraudulent accounts.

4. System Architecture

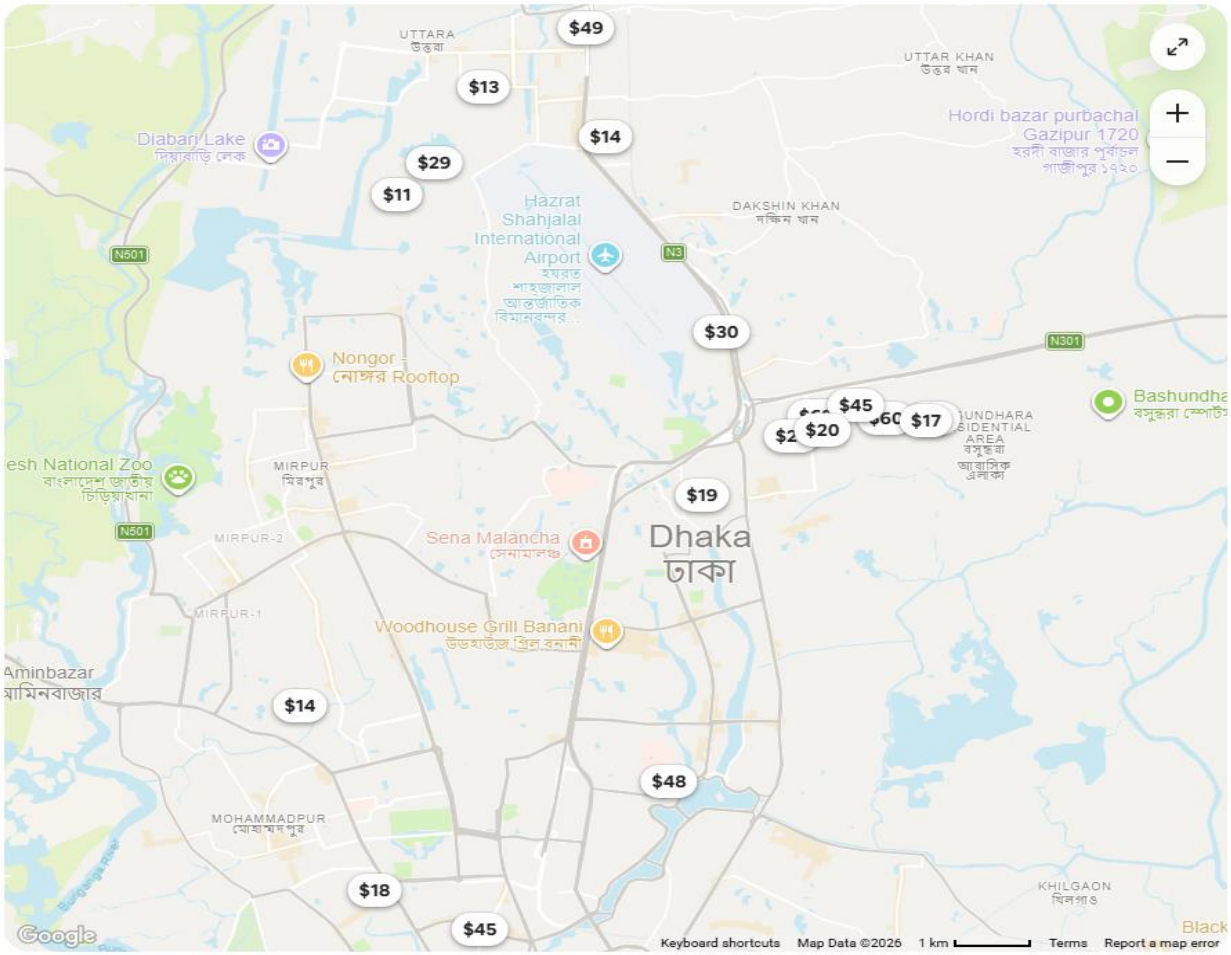
4.1 System Diagram

- Frontend → React.js
- Backend → Node.js + Express
- Database → MySQL (XAMPP)



4.2 Map

Map will be used for booking based on the nearest location, will preview the price of the house and etc.



5. Data Requirements

5.1 Logical Data Model

5.1.1 Main Entities

- Users
- Listings
- Bookings
- Reviews
- Payments

5.1.2 Relationships

- One User → Many Listings (Host)
- One User → Many Bookings (Guest)
- One Listing → Many Bookings
- One Booking → One Payment

5.2 Data Dictionary

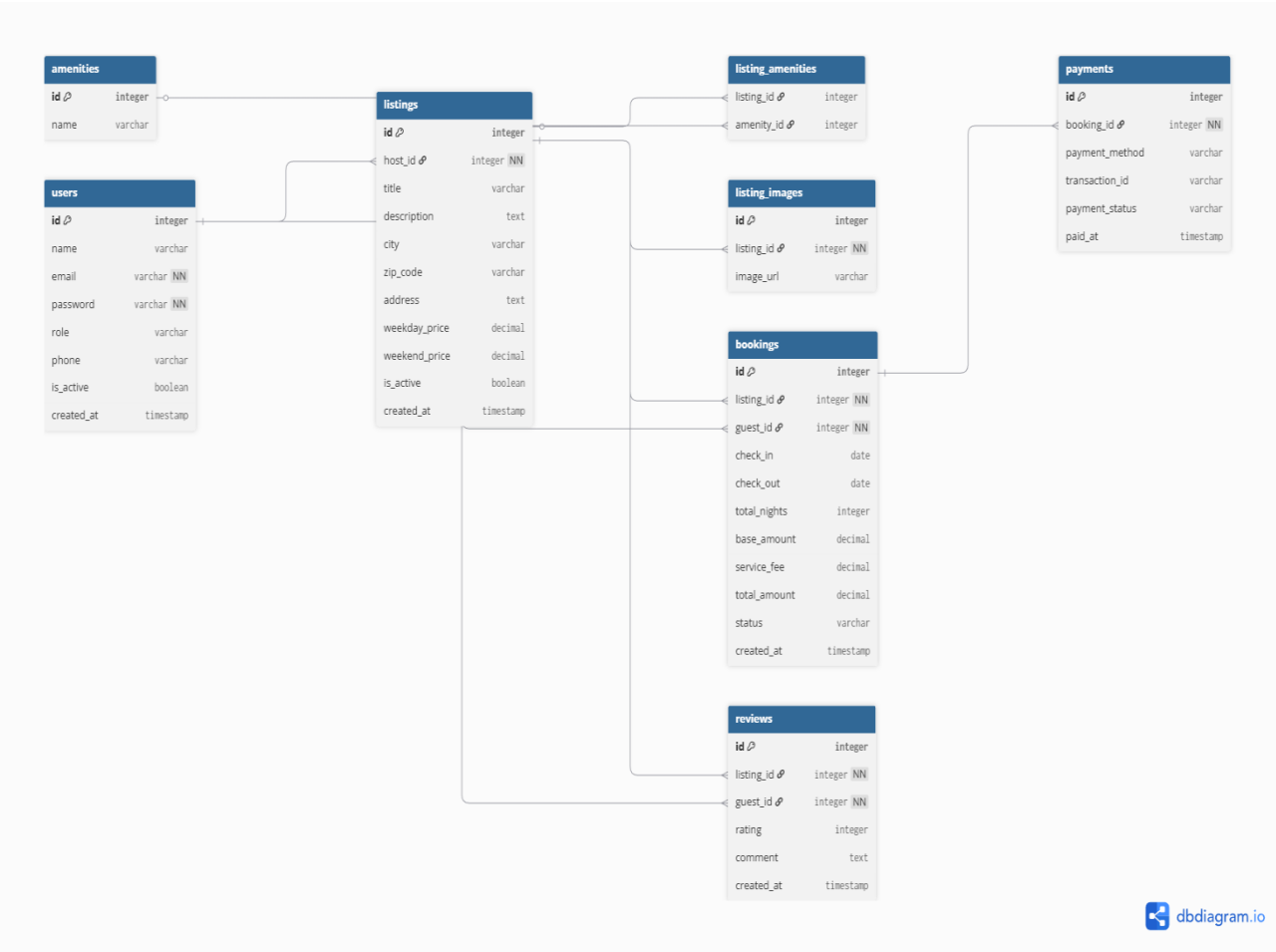
5.2.1 User Table

Field	Type	Description
user_id	INT(PK)	Unique ID
name	VARCHAR(100)	Full Name
email	VARCHAR(100)	Unique Email
password	VARCHAR(100)	Hashed
role	ENUM	Guest/Host/Admin

5.2.2 Listings Table

Field	Type
listing_id	INT(PK)
host_id	INT(PK)
title	VARCHAR(255)
city	VARCHAR(100)
weekday_price	ENUM
weekend_price	DECIMAL
is_active	DECIMAL

5.3 Database Schema



5.4 Reports

- Monthly Revenue Report
- Host Earnings Report
- Booking History Report
- User Activity Report
- Reports include
    - Sorting by date
    - Total calculations
    - Export to CSV (optional)

5.5 Data Acquisition, Integrity, Retention, and Disposal

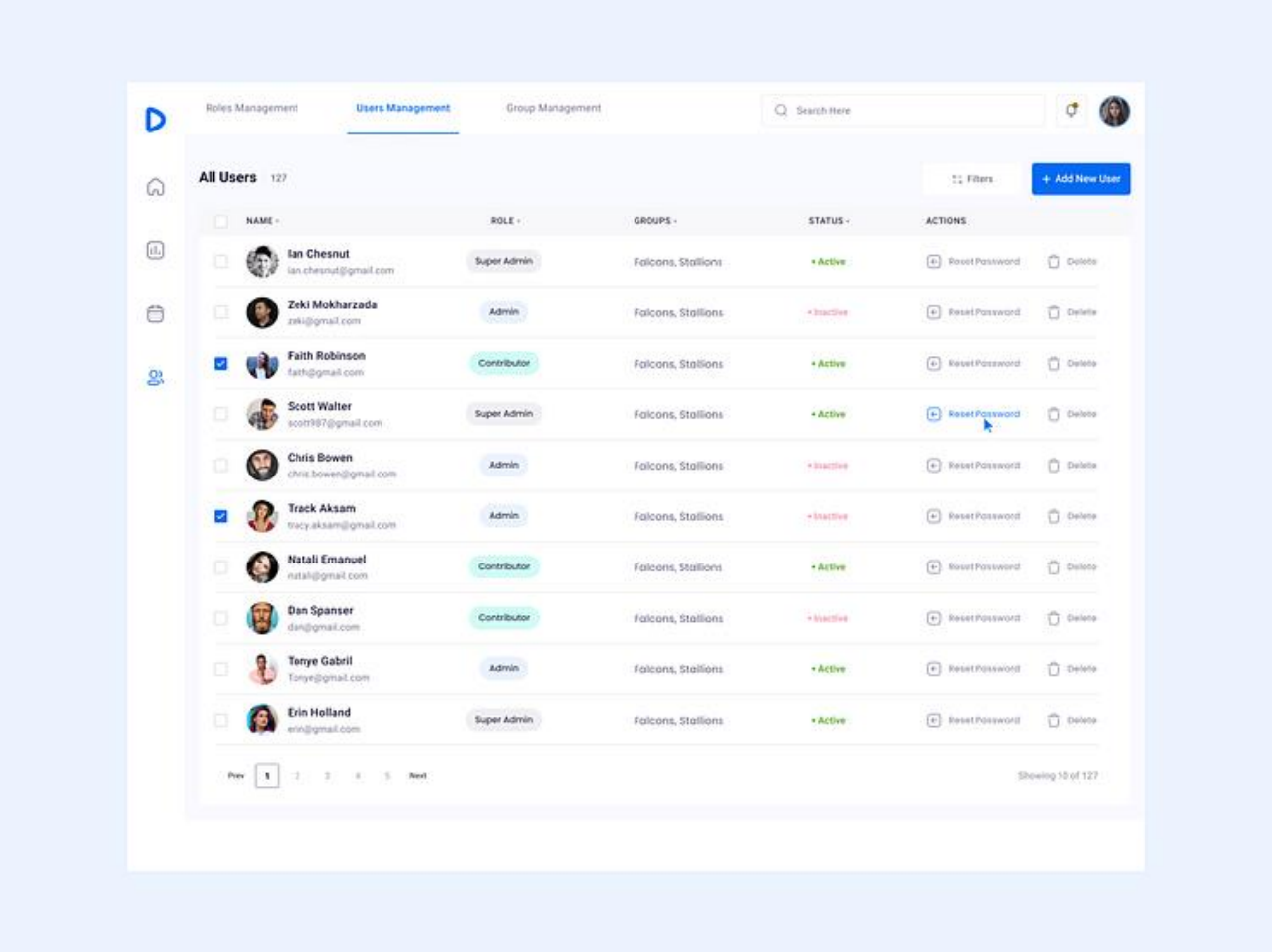
- Foreign Key constraints enforced
- Daily database backup required
- Deleted data stored for 30 days (soft delete)
- Encryption for sensitive data

6. External Interface Requirements

6.1 User Interfaces

- Responsive design
- Dashboard for Host & Admin
- Error messages clearly displayed
- Form validation on client and server

6.1.1 Admin Dashboard UI



6.2 Software Interfaces

- Payment Gateway API
- Email Notification Service
- RESTful JSON communication

6.3 Hardware Interfaces

No specialized hardware required.
Supports desktop, laptop, mobile.

6.4 Hardware Interfaces

- HTTPS protocol required
- REST APIs using JSON
- Secure session handling using JWT

6.5 Communications Interfaces

- HTTPS protocol required
- REST APIs using JSON
- Secure session handling using JWT

7. Quality Attributes

7.1 Usability

- **NFR-1:** System must be easy to navigate.
- **NFR-2:** Booking process ≤ 5 steps.
- **NFR-3:** Response error messages user-friendly.

7.2 Performance

- **NFR-4:** Page load time ≤ 3 seconds.
- **NFR-5:** Search results load ≤ 2 seconds.
- **NFR-6:** System supports 1000 concurrent users.

7.3 Security

- **NFR-7:** Password hashing mandatory.
- **NFR-8:** Role-based access control.
- **NFR-9:** Protection against SQL injection.
- **NFR-10:** Use HTTPS encryption.

7.4 Reliability

- **NFR-11:** System uptime 99%.
- **NFR-12:** Automatic error logging.

7.5 Scalability

- **NFR-13:** Database must support horizontal scaling.
- **NFR-14:** APIs must be stateless.

8. Internationalization and Localization Requirements

- Currency symbol configurable
- Date format supports multiple formats (DD/MM/YYYY, MM/DD/YYYY)
- Multi-language ready structure
- Time-zone awareness for bookings

9. Other Requirements

- Audit logging for all financial transactions
- GDPR-like data protection compliance
- Email verification required during registration
- Admin monitoring dashboard

10. Future Improvements

10.1 Mobile Application

- Native Android and iOS mobile applications.
- Push notification support for bookings and messages.
- Offline browsing of previously viewed listings. -friendly.

10.2 Integrated Online Payment Gateway

- Full integration with secure third-party payment gateways (e.g., Stripe, PayPal).
- Refund processing system.
- Partial payment and installment booking options.
- Wallet system for Hosts and Guests.

10.3 Real-Time Messaging System

- In-app chat between Host and Guest.
- File/image sharing capability.
- Message notification alerts.

Appendix A: Glossary

Term	Meaning
Host	Property owner
Guest	Renter
Booking	Reservation record
Service Fee	Platform commission
API	Application Programming Interface
JWT	JSON Web Token

Appendix B: Analysis Models

- ER Diagram
- Use Case Diagram
- System Architecture Diagram
- Data Flow Diagram (DFD)
- Sequence Diagram (Booking Process)